DBMS

* **Data abstraction:** Hiding of unnecessary things from users is called data abstraction.

Three levels of abstraction:

(1) **Physical level:** How data is actually stored in database.

(2) **Logical level:** What data is stored in database.

(3) **View level:** Describes how user interact with database.

* **Schema:** Logical container or structure that organizes and define the structure of database.

* **DBMS Architecture:** Defines how the various components of the system work together to store, manage and retrieve data efficiently.

* **1-Tier Architecture:** The entire database application including the user interface, storage, application logic resides on a single machine or computer. Not for multiple users.

E.x. sqlite

Notes

03 May Friday 2024

MAY 2024

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

124-242 Week 18

* 2-Tier: Client (frontend) Database server (backend)

The presentation layers run on a client & data is stored on a server.

* 3-Tier: It seperates the application into three logically distinct layers presentation, application and data layer.
* Presentation: User interface
* Application: Business logic - server
* Data: Manages data storage & processing
e.x. Database server

* Referential Integrity in Foreign key: Ensures that the relationship between tables remain accurate, consistent and meaningful within a relational database.

* Integrity constraints: Used to prevent invalid data from entering database.

(1) Entity Integrity: Each record in the table must be uniquely identified by a primary key.

(2) Domain Integrity: Ensures the validity and appropriateness of data values within a specific column or attribute of a table.

Notes

Domain Integrity:

```
create table students (
    id int,
    age int check(age>=18)
);
```

May Saturday 2024 | 04

Week 18 | 125-241

(3) Referential Integrity: Ensures that the foreign key column matches the corresponding primary key column in another table.

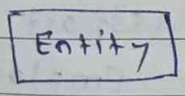
(4) Key Constraints: Ensures uniqueness for the primary key.

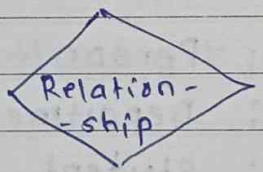
check, Null, Unique, Default constraints.

* Super key: The set of one or more attributes that can uniquely identify a tuple.

* Candidate key: The minimal set of attributes that can uniquely identify a tupe is known as a candidate key.

* ER model: Its primary role is to offer a visual representation of a database's architecture by illustrating the entities, their respective attributes & the interconnections between them.

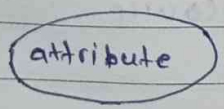


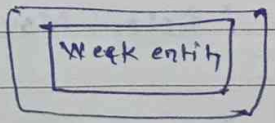


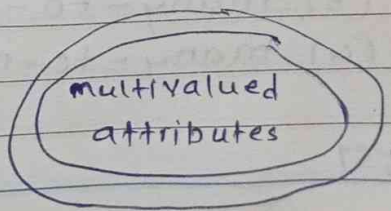
attribute to entity relationship

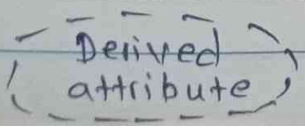
Sunday 05

Notes









Ex. of weak entity → strong Entity → Employee
Weak Entity → child

06 May Monday 2024

How we uniquely identify a child:
we use a composite key:
(empid + child attribute)

127-239

empid → comes from strong entity.

MAY 2024

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

Week 19

* Weak entity: It doesn't have a primary key. It relies on a related strong entity (known as the 'owner' entity for its identity.

* Simple attribute: Atomic & indivisible further.
Composite, single valued, multivalued attributes.
* Derived attributes: Derived from other attributes within the database.

* Strong Relationship: A strong relationship exists when two entities are highly dependent on each other and one entity cannot exist without the other.

* Weak Relationship: Two entities are related but one entity can exist without the other.

* Degree in DBMS: The no. of attributes/columns that a relation/table has.

* Cardinality — Types of relationships:

(1) One-to-one : Person → Passport
(2) one-to-many : Department → Employee
(3) many-to-one : Student → Course
(4) many-to-many : Actor : Movie

Notes

337

* Normalisation: The process of organizing the data to reduce redundancy & improve data consistency by dividing a database into two or more tables.

When we have same data for some set of columns, it lead to different anomalies.
It also increases the size of database.

① Insertion Anamoly: Occurs when it is difficult to insert data into database due to the absence of other required data.

id | name | age | dept | manager | salary

Here, anamoly arises when we want to add a new department but there is no employee in that dept yet.

② Deletion Anamoly: Occurs when deleting data removes other valuable data.

If we delete all the record in the table, you will loose the track of dept, their manager & salaries.

Notes

(3) Updation Anamoly: occurs when changes to data require multiple updates.

changing the salaries of people working in HR department, we need to update it all all plaies where dept = HR.

id | name | age | dept | manager | salary

id | name | age | dept dept | manager | salary

* Types of normalisation:

1NF, 2NF, 3NF, BCNF

* Functional Dependency: Describes the relationship between two attributes in a relation.

X (determinant) ⟶ Y (dependent)

Y is functionally dependent on X.

or

X determines Y

For every given value of X, we can uniquely identify Y.

Notes

* Properties of FD: (Armstrong's Axioms)

(1) Reflexivity: $Y \subseteq X$, then $X \rightarrow Y$

$\{A, B\} \rightarrow A$, then $A \rightarrow B$

(2) Augmentation: If $X \rightarrow Y$, then $XZ \rightarrow YZ$

(3) Transitivity: $X \rightarrow Y$, $Y \rightarrow Z$ then $X \rightarrow Z$

(4) Union: $X \rightarrow Y$, $X \rightarrow Z \Rightarrow X \rightarrow YZ$

(5) Decomposition: $X \rightarrow YZ$ then $X \rightarrow Y$, $X \rightarrow Z$

* Types of FD:

(1) Trival: $XY \rightarrow X$ (X is a subset of Y)

(2) Non-Trival: $X \rightarrow Y$ (Y is not a subset of X)

* Attribute Closure: The closure of an attribute set X (X^+) is:

All attributes that can be functionally determined from X using the given F.Ds

F.x. $R(A, B, C, D)$ $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$

A^+ : $A^+ = \{A\}$

$= \{A, B\}$ .... $A \rightarrow B$

$= \{A, B, C\}$ ..... $B \rightarrow C$

$= \{A, B, C, D\}$ .... $C \rightarrow D$

Notes

$B^+ = \{B, C, D\}$ $C^+ = \{C, D\}$ $D^+ = \{D\}$

* Super key: set of all attributes whose closure contains all the attributes given in a relation.

$A^+ = \{A, B, C, D\}$

∴ A is a super key

* candidate key: the minimal set of attributes whose closure contains all the attributes of the relation.

---

* First normal Form (1-NF):

A relation is in 1NF if all attributes contains only atomic (indivisible) values and there are no repeating groups.

No.	name	phone
101	---	9767, 78910

↑ multiples in one cell

∴ It violates 1NF.

---

* 2 NF: A relation is in 2 NF if

(i) It is in 1 NF.

(ii) It has no partial dependency.

Notes

Partial dependency: No non-prime (not part of candidate key) attribute is dependent on a part of candidate key.

* We have to check for 2NF only when candidate key is composite (more than one attribute).

* If it is single attributed, table is already in 2NF.

E.X. R(A, B, C, D) $AB \longrightarrow C$ $A \longrightarrow D$

$AB^+ = \{A, B, C, D\}$

$A^+ = \{A, D\}$

$B^+ = \{B\}$

∴ AB is the candidate key.

$A \longrightarrow D \Longrightarrow$ D only depends on A & not B (A is a part of candidate key)

∴ Relation is not in 2NF.

* How to convert a relation in 2NF?

R(A, B, C, D) $AB \longrightarrow C$, $AB \longrightarrow D$, $B \longrightarrow C$

Partial dependency : $B \longrightarrow C$

create a new relationship for partial dependency

R1(B, C)

Remove C from original relation

R2(A, B, D)

Notes

* 3NF: A relation is in 3NF if:
(i) It is already in 2NF
(ii) No transitive dependency exists.

Transitive dependency: No non-prime attribute should be dependent on another non-prime attribute.

E.x. $A \longrightarrow B$ $B \longrightarrow C$

A: candidate key

If B is not a candidate key, then both B & C will be non-prime

A relation is in 3NF if for every FD:

$X \longrightarrow Y$

At least one condition is true:

* X is a super key
* Y is a prime attribute (part of some candidate key)

E.x. $X \longrightarrow Y$, $X \longrightarrow Z$, $Z \longrightarrow A$

$X^+ = \{X, Y, Z, A\}$

∴ X is the candidate key

∴ Non prime attributes: $\{Y, Z, A\}$

Notes

Now $X \longrightarrow Z$, $Z \longrightarrow A$

A depends on Z and not directly depend on X which is the candidate key

∴ This relation is not in 3NF.

To remove this dependency: Decompose

$R1(X, Y, Z)$ $R2(Z, A)$

★ BCNF: (Boyce Codd Normal form)

BCNF removes a special case the 3NF still allows.

A relation is in BCNF if for every functional dependency $X \longrightarrow Y$,

X must be a super key.

E.x. $AB \longrightarrow C$ $C \longrightarrow B$

$AB^+ = \{A, B, C\}$

∴ AB is candidate key.

Check 3NF

$C \longrightarrow B$

C is not super key ✗

But B is prime ✓

So relation is in 3NF. ✓

Check BCNF

$C \longrightarrow B$

C is not a super key

∴ Relation is not in BCNF.

Notes

After 3NF, if we still see
$X \longrightarrow Y$ where X is not a super key
$\Longrightarrow$ decompose

* Dependency Preserving Decomposition:
A decomposition is dependency preserving if all original functional dependencies can be enforced by looking at the decomposed tables individually.

* Lossy decomposition: When a relation R is decomposed into two or more relations, but data is lost & the original relation R can't be reconstructed by joining these decomposed relations.

* Lossless: A decomposition is lossless if the natural join of decomposed relations gives exactly the original relation.

* Minimal cover: It is a simplified version of the original set of functional dependencies.

$A \longrightarrow BC$, $B \longrightarrow C$, $A \longrightarrow B$, $AB \longrightarrow C$

Minimal cover;

$\{A \longrightarrow B, B \longrightarrow C\}$

Notes

* Transaction: A logical unit of work that comprises one or more database operations (like read, write, commit, rollback).
It is a sequence of operations treated as one single unit.

ACID Properties: Ensures reliable, safe and consistent database transactions.

(1) Atomicity: If any part fails, the whole transaction is rolled back.

(2) Consistency: It ensures that the database remains in a consistent state before and after the execution of each transaction.

E.x. A has 100 Rs, B has 100 Rs
A transfers 50 to B
∴ A has 50 Rs & B has 150 Rs

∴ Before transaction : Total = 200 Rs
After transaction : Total = 200 Rs

* The database rules should be followed:
E.x. Account balance cannot be negative.

Notes

If rule breaks, transaction is aborted.

(3) Isolation: It ensures that if there are two transactions A & B, then the changes made by transaction A should not be visible to Transaction B until transaction A commits.

E.x. Account A = 10,000

T1: A ⟶ B 7000

T2: A ⟶ C 5000

Without isolation, lost update / Dirty concurrency problem may occur.

* How DBMS fix this problem using Isolation:

The DBMS does not truly run the transactions at the same time on the same data

It uses ⟶ Locks, timestamp, Serialization

T1 completes first

A: 10000 —B→ 10000 - 7000 = 3000

A: 3000 —C→ 5000 > 3000 ⟹ Abort

∴ Only one valid transaction succeeds the other is rolled back.

∴ Database

Notes

* Dirty Reading: Reading uncommitted data from another transaction.

(4) Durability: Once committed, data will not be lost even if system crashes.

Most DBMS use a technique called Write-Ahead logging (WAL) to ensure durability.
Before modifying data in the database, the DBMS writes the changes to a transaction log (often stored in disk) in a sequential manner.

* Isolation level: It determines the degree to which the operations in one transaction is isolated from other transactions.

* Anamolies / Voilations to isolation level:

(1) Dirty Read: Reading data written by a transaction thas has not yet committed.

(2) Non-Repeatable Read: Reading the same row two times within a transaction and getting different values because another transaction modified the row and committed.

(3) Phantom read: Getting different sets of rows in subsequent queries within the same transaction because another transaction inserted or deleted row and commit.

* Schedule: The sequence in which a set of concurrent / multiple transactions are executed.

* Incomplete schedule: An incomplete schedule is one where not all transactions have reached their final state of either commit or abort.

T1	T2
R(A)	
W(A)	
	R(B)
	W(B)
	commit

T1 is still in process as there is not commit

* Serial schedule: A serial schedule is one where transactions are executed one after other.

T1	T2
R(A)	
W(A)	
commit	
	R(B)
	W(B)
	commit

T2 starts only after T1 is completed / committed.

Notes

* Non-Serial / concurrent schedule: Multiple transactions can execute simultaneously.

T1	T2	T3
R(A)		
	R(A)	R(B)
	W(A)	
commit		
		commit

T2 doesn't need to wait for T1 to commit, it can start at any point.

* Conflict Serializable schedule:

A schedule is conflict serializable if we can convert it into a serial schedule by swapping non-conflict operations.

Two operations conflicts if:

(i) They belongs to different transactions.
(ii) They access the same data item.
(iii) Atleast one of them is write (W).

Notes

Isolation level: It determines the degree to which the operations in one transaction is isolated from other transaction.

* Anamolies / Violations to isolation level:

(1) Dirty Read: A transaction reads uncommitted data from another transaction.

E.x. Account A = 10,000

Transaction T1: A ⟶ B 7000 → uncommited

Transaction T2: A ⟶ C

T2 reads A's balance as 10,000 ⇒ Dirty Read

(2) Non-Repeatable Read: A transaction reads the same row twice and gets different values.

A = 1000

Transaction T1 ⟶ Reads ⟶ 1000

Transaction T2 ⟶ Update +500 ⟶ 1500 → commits

same query different result

Transaction T1 again ⟶ 1500

∴ This is Non-Repeatable Read.

(3) Phantom Read: A transaction re-executes a query and finds new rolos (phantoms) added by other transaction.

Transaction T1:
select * from Employees where salary > 25000;
⟶ Gave 5 rows.
Transaction T2: Inserts new employee with salary 30000.
Transaction T1 again:
select * from employees where salary > 25000
⟶ Result: 6 rows ⟹ A new row appeared suddenly.
∴ This is Phantom Read.

* To handle these anamolies, DBMS uses transaction isolation levels:
(1) Dirty Read: A transaction can only read committed data.
(2) Non-Repeatable Read: Once a row is read ⟶ it gets locked. Other transactions cannot modify it until current transaction ends.
(3) Phantom Read: Prevents insert / delete affecting your query.

* Conflict Serializable Schedule: A schedule is conflict serializable if we can convert it into a serial schedule by swapping non-conflict operations.

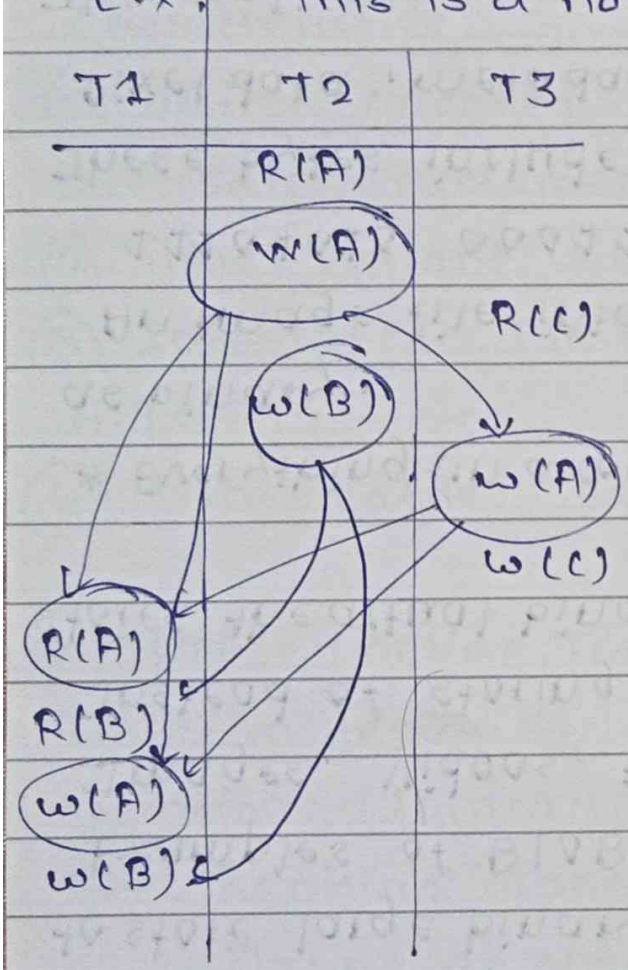
* Two operations conflicts if:

(1) They belongs to different transactions AND

(2) They access the same data item. AND

(3) Atleast one of them is write.

Ex. This is a non-serial schedule



To check if this schedule is serializable or not?

* Find conflicts:

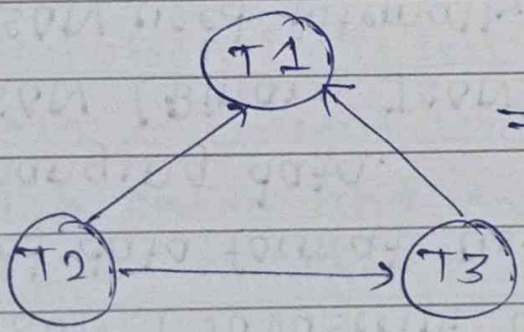
W2(A) happens before W3(A) $\Longrightarrow$ T2 $\longrightarrow$ T3

W3(A) happens before R1(A) & W1(A)

T3 $\longrightarrow$ T1

W2(B) happens before R(B) & W1(B)

$\therefore$ T2 $\longrightarrow$ T1



$\Longrightarrow$ No cycle exists

$\therefore$ The schedule is conflict serializable.

Serial order $\longrightarrow$ T2 $\longrightarrow$ T3 $\longrightarrow$ T1

* JSON (Javascript Object Notation): A text-based data format used for storing and exchanging data.
* BSON (Binary JSON): A binary-encoded format of JSON used internally by MongoDB, for efficient storage & processing.
* BLOB (Binary Large Object): A data type used to store large binary data in database.
Examples of BLOB Data:
Images, Videos, Audio, PDFs
Instead of storing image.jpg, database stores the actual binary content of the image.

* Everything in a computer is ultimately stored as binary.
An image file internally looks like:
11101010 00011010 .....
These bytes include: File header (JPEG, PNG)
Pixel data, metadata
This entire sequence of bytes is what gets stored in a BLOB.

* Used in SQL databases like MySQL, Oracle

* Data Warehouse: A data warehouse is a system used to store large amounts of historical data from different sources, optimized for analytisis and reporting, not day-to-day transactions.

Characteristics:

1. Subject-oriented: Data is organized by subjects like: sales, customers, products.
2. Time-Variant: stores historical data.
3. Integrated: combines data from: Databases, APIs, logs, etc.
4. Non-Volatile: Data is mostly read-only.

* OLTP: (Online Transaction Processing): A system designed to handle day-to-day operations (transactions) quickly and accurately. E.x. Banking transactions.

Placing an order on an e-commerce site, etc.

Characteristics:

* Handles many small transactions.
* Operations: Insert, Update, delete.
* Data is current and detailed.
* High speed and concurrency.
* Tools: MySQL, PostgreSQL.

* OLAP: Online Analytical Processing
A system designed for analytics, reporting and decision-making.

* Characteristics: * Handles complex queries.
* Uses historical data.
* Optimized for analysis.
* Tools: Microsoft Redshift, Google BigQuery.

* OLTP: → Systems feed data into a data warehouse
* OLAP: → Systems run on top of that warehouse

* Data WareHouse: → Used to store structured, historical data for analysis & reporting (OLAP)

* Data Center → A physical facility that houses → servers, storage systems and networking equipments.
It is used to run applications, store data and provide computing power.

* Query Optimization: To improve the efficiency of a query by making it execute faster and consume less resources.

(1) Indexes: An index is a data structure that helps the database find rows faster without scanning the entire database.

Select * from students where roll_no = 42;
⟹ Scans entire document ⟹ O(n) time comp.

Create index idx_roll on students(roll_no)
⟹ Uses a search tree (B tree) ⟹ O(log n)

Notes

Types of Indexes:

(1) Primary: Automatically created on primary key

(2) composite: create index idx_name_city
on customers(name, city);
useful when queries filter on both columns.

* When indexes can hurt performance:
Insert, update, delete ⟹ Because DBMS must update the index too.

(2) Selecting only necessary columns insted of '*';
(3) Partitioning large tables.
(4) Cache results: Cache the results of frequently executed queries to avoid redundant calculations.

* Tertiary storage: Optical disk and magnetic tape used for data backup.

* B-Tree (M-Way tree): Self-balancing tree data structures that maintain soited data & allow searches, access, insertion, deletion in logarithmic time.

* All leaf nodes are ~~th~~ at the same level.
* can have minimum 2 children and max x children

Notes

* Data masking: Hiding or changing sensitive information so it's protected from unauthorized access. This allows the data to be used for testing, analytics, training without exposing the real data.

Techniques: Substitution, shuffling, nulling out

* NoSQL (Not only SQL):
(i) Non-tabular databases, schema free.
(ii) Provides flexible schemas and scale easily with large amounts of data and high user loads.

Advantages:

(1) Flexible schema: In RDBMS, issue comes when we do not have all the data with us or we need to change the schema which is a huge task on the go.

(2) High Availability: If a server fails, we can access data from another server as well, as in NoSQL data is stored at multiple servers.

(3) Queries for insert/read operations:
Queries in NoSQL databases can be faster than SQL.
(i) Data in SQL databases is typically normalized, so a single query may require to join data from multiple tables. As tables grow in size, the joins can become expensive.

Notes

(ii) When using NoSQL, data to access together should be stored together, Queries typically do not require joins, so the queries are very fast.

* When to use NoSQL: (able to move quickly & easily)
  (1) Fast-paced agile development
  (2) Huge volumes of data.
  (3) Modern application paradigm like micro-services & real-time ~~str~~ streaming.

* NoSQL databases don't support ACID transactions. while some

* Types of NoSQL Data models:

* (1) Key-value stores: Every data element in the database is stored as a key value pair,
(2) Use cases: shopping carts, user profiles
(3) E.x. Oracle NoSQL, Amazon DynamoDB

(2) Column-Oriented: Data is organized as a set of columns instead of rows.
  * Use cases includes analytics
  * E.x. cassandra.

Sunday 26

Notes

(3) Document Based stores: Stores data in documents similar to JSON (Javascript - Object Notation) objects.
* Each document includes pair of fields & values.
* Supports ACID properties hence suitable for transactions.
* E.x. MongoDB.

(4) Graph Based Stores: Each element is stored as a node (such as a person in a social media graph) The connections between elements are called links or relationships.

---

* Dis-advantages:

(1) Data Redundancy: Data models in NoSQL databases are typically optimised for queries and not for reducing data redundancy.
* They can be larger than SQL databases.
* Storage is currently cheap, hence this data redundancy is considered a minor drawback.

(2) Update & Delete operations are costly
(3) Doesn't support ACID properties except MongoDB
(4) Doesn't supports Data entry with consistent constraints.

Notes

* Types of Databases:

(1) Relational Database

(2) Object-oriented database.

(3) NoSQL Database

(4) Hierarchical Database: A parent record can have several child records but each child record can only have one parent record.

(5) Network Database: Extension of hierarchical databases.

The child records are given the freedom to associate with multiple parent records.

Organised in a Graph structure.

---

* Clustering in Databases: (Replica-sets)

(i) The process of combining one or more servers or instances connecting a single database.

(ii) Sometimes one server may no be adequate to manage the amount of data or the no. of requests, that is when data cluster is needed.

Advantages:

① Load Balancing: Allocating the workload among different servers that are part of the cluster.

*This indicates that if a huge strike in the traffic appears, there is a high assurance that it will be able to support the new traffic.

Notes

* This can provide scaling seamlessly as required.
* Without load balancing, a particular machine or server could get overworked & traffic would slow down.

② High Availability: The amount of availability greatly depends on no. of transactions the database is running.

With data clustering, we can reach extremely high levels of availability due to load balancing.

---

* Partitioning in DB Optimization:

(i) Large table is divided into smaller, mangeable pieces called partitions, while still appearing as a single table to the user.
(ii) If we can apply partitioning to the database that is already scaled out i.e. equipped with multiple servers, we can partition our database among those servers and handle the big data easily.

* Types of Partitioning:

① Range Partitioning:

Notes

2024 data ——→ Partition 1
2023 data ——→ Partition 2

* Clustering: Same data across all servers (redundancy)
* Distributed DB: Data split across multiple servers (nodes)

(2) List Partitioning:

Pune ⟶ Partition 1

Mumbai ⟶ Partition 2

* Horizontal Partitioning: Slicing relation horizontally. Independent chunks of data tuples are stored in different servers.
* Vertical Partitioning: Slicing relation column-wise Need to access different servers to get complete tuples.

* When to apply Partitioning:
(i) Database becomes much huge that managing and dealing with it becomes a tedious task.
(ii) The no. of requests are enough larger that the single DB server access is taking huge time & hence the system's response time become high.

* Distributed database: A single logical database that is spread across multiple locations (servers) and logically interconnected by network.
This is the product of applying DB optimization techniques like clustering, Partitioning and sharing.

Notes

31 May Friday 2024

MAY 2024

S	M	T	W	T	F	S
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

152-214 Week 22

* sharding: Large database is split into smaller pieces called shards and shored across multiple servers.
* Each shard contains a portion of data and together they form the complete dataset.

Working: (1) A shard key is chosen (user_id, region, email, etc).
(2) A rule decides where data goes
(3) The application or database router sends queries to the correct shard.

E.x. shard = user_id % 3
user_id = 7 ⟶ goes to shard 1
user_id = 8 ⟶ goes to shard 2
user_id = 9 ⟶ goes to shard 0

Now queries run faster because each server handles fewer rows.
Load is distributed.

Horizontal Partitioning	sharding
* Splitting a table into multiple parts by rows.	* Splitting data by rows
Notes	* But storing each part on different servers.
* Usually done within the same database server.	* Requires routing layers.
* Managed internally by DBMS.	

01 June Saturday 2024

JUNE 2024

S	M	T	W	T	F	S
30						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

153-213

Week 22

CAP (Consistency, Availability, Partition Tolerance) Theorem / Brewer's theorem:

A distributed database can guarentee at most two of the three properties at the same time.

(1) Consistency:

* Every read receives the most recent write or an error.
* All nodes see the same data at the same time.

E.x.
You update account balance.
Immediately after this, every server returns the updated balance.

(2) Availability:

* Every request receives a response (success or failure).
* The system remains operational even if some nodes fail.

(3) Partition Tolerance: The system continues to function even when network failures occur between nodes.
Nodes may not be able to communicate temporarily.

02 Sunday Partition tolerance is essential in real distributed systems because network failures are unavoidable.

Notes

* Why Can't we have all three ?

A network partition occurs. some nodes cannot communicate.

Now the system have to choose:

Choice 1 : Maintain Consistency:

Reject some requests until node sync.

choice 2 : Maintain Availability

Allow reads/ writes on both sides.

Data may become inconsistent temperarily.

* Important Real-World Insight:

* Partition tolerance is mandatory in real distributed databases.

* So systems usually choose between : CP, AP

If the system is :

CP : One side stops accepting writes to keep data correct.

AP : Both sides accept writes, later resolve conflicts.

Notes

04 June Tuesday 2024

JUNE 2024

S	M	T	W	T	F	S
30						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

156-210 Week 23

* Master-Slave Database concept:

(Primary Replica Replication)

A database architecture where:

* one server is master (Primary)
* one or more servers are slaves (Replicas)
* Master handles write operations.
* Slaves handles read operations by keeping copies of the master's data.

* After write operation, master sends the updated data to slave servers and waits until slaves confirm the update.
* If a master fails, a slave can be promoted to master.
* Data is not lost because copies exist.